University of Sheffield

COM3529 Software Testing and Analysis

Test Automation & Test Scope

Dr José Miguel Rojas

What do you understand by the phrase *automated test*?

JUnit example

```java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class TriangleTest {

    @Test
    public void shouldClassifyEquilateral() {
        Triangle.Type result = Triangle.classify(10, 10, 10);
        assertEquals(Triangle.Type.EQUILATERAL, result);
    }

    // ...
```

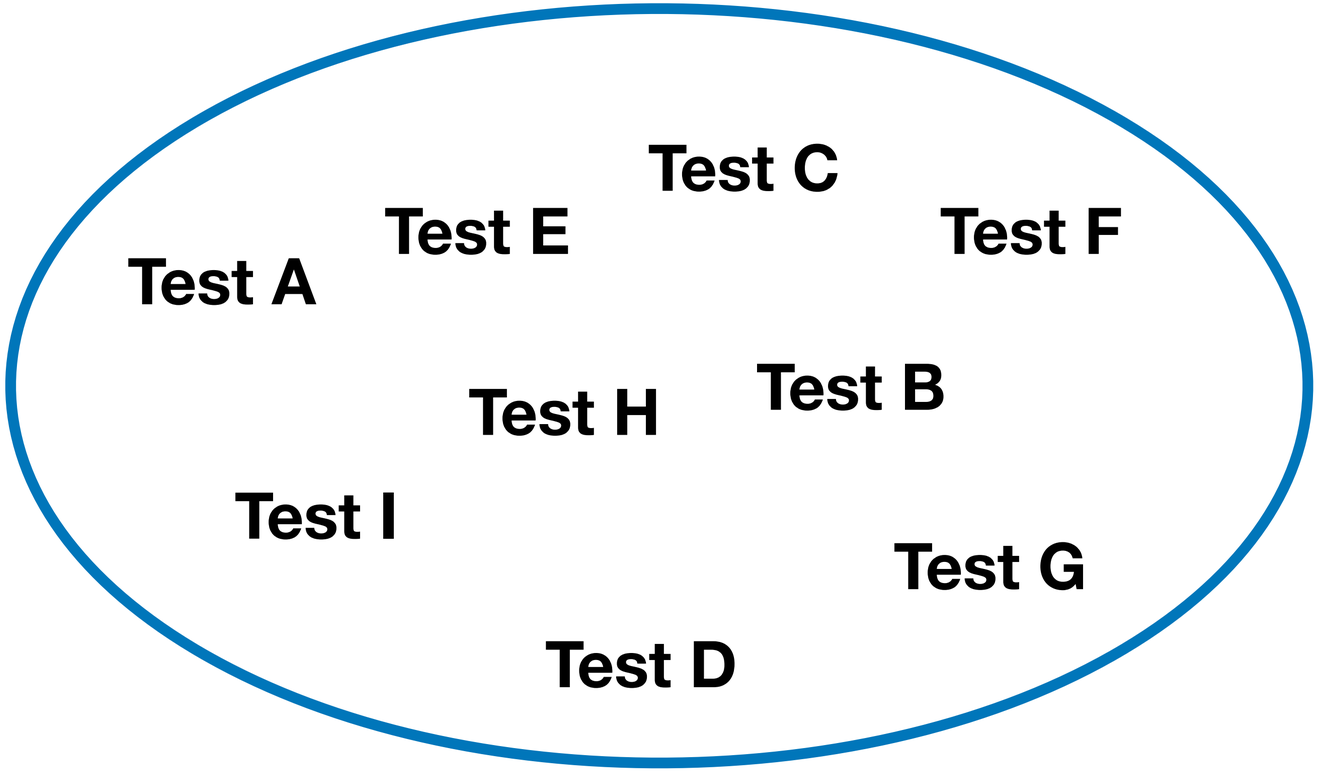
RSpec example

```
require_relative "../spec_helper"

describe "the add page" do
  it "is accessible from the search page" do
    visit "/search"
    click_link "Add a new player to the database"
    expect(page).to have_content "Add Player"
  end

  it "will not add a player with no details" do
    visit "/add"
    click_button "Submit"
    expect(page).to have_content "Please correct the errors below"
  end

  it "adds a player when all details are entered" do
    add_test_player
    expect(page).to have_content "George Test"
    clear_database
  end
end
```

Ingredients of an Automated Test Case

1. The inputs needed to put the software into the right state for the test
2. The actual test case inputs
3. The expected results of the test
4. Reset of the system state

A Test Suite – A Set of Tests



Ideally, the tests can be executed in any order

The Dawn of Test Automation

Testing has always been part of programming

… when you wrote your first program, you almost certainly tried it out with some sample data

For a long time, this was the state of the art in industrial practice!

In the early 2000s, software development practices started to change

Software systems got **too big** and **too complex** for manual testing to remain an effective and efficient way to ensure they were working and **remained working**

Testing at the Speed of Modern Software Development

Software systems are growing larger and evermore complex.

A typical application or service at Google, for example, is made of thousands or millions of lines of code.

The ability for humans to **manually validate every behaviour in a system** has been **unable to keep pace with the explosion of features and platforms in most software**.

Testing at the Speed of Modern Software Development

Imagine what it would take to manually test the functionality of Google search – every time the code was changed.

… not just web search, but images, flights, movie times, etc.

Then multiply that for every language, country, and device that must be supported.

Then add in factors like accessibility and security.

Manual testing does not scale. We need automation.

Developer-Driven Automated Testing

The idea of coding automated tests (e.g., in JUnit) as a means of improving productivity and velocity may seem antithetical.

After all, the act of writing tests can take just as long (if not longer!) than implementing a feature in the first place … right?

On the contrary!

In industry, investing in software tests provides **multiple benefits** to developer productivity.

Less Debugging

Tested code has fewer defects when it is pushed to production.

Crucially, it also has fewer defects throughout its existence – since code tends to be updated during its lifetime.

> … it will be changed by other teams and even automated code maintenance systems.
>
> Changes to code, or its dependencies, can be quickly detected by an automated test and rolled back before the problem reaches production.

Increased Confidence in Changes

Projects with good tests can be modified with confidence since all the important behaviours are continuously being verified.

These projects *encourage* refactoring.

Regression Testing – After a change, we can re-run the automated tests to ensure we didn't break any of the existing functionality.

More on this later in this module…

Improved Documentation

Software documentation is notoriously unreliable!

Clear, focused tests that exercise one behaviour at a time function as executable documentation.

Thoughtful Design

Writing tests for new code is a practical means of exercising the API design of the code itself.

If new code is difficult to test, it is often because the code being tested has too many responsibilities or difficult-to-manage dependencies.

Well-designed code should be modular, avoiding tight coupling and focusing on specific responsibilities.

Fixing design issues early means less rework later.

Fast, High Quality Releases

With a healthy automated test suite, teams can release new versions of their application with confidence.

Many large projects, involving hundreds of engineers and thousands of code changes submitted every day, involve very short release cycles – often every day.

This would not be possible without automated testing.

Benefits of an Automated Test Suite

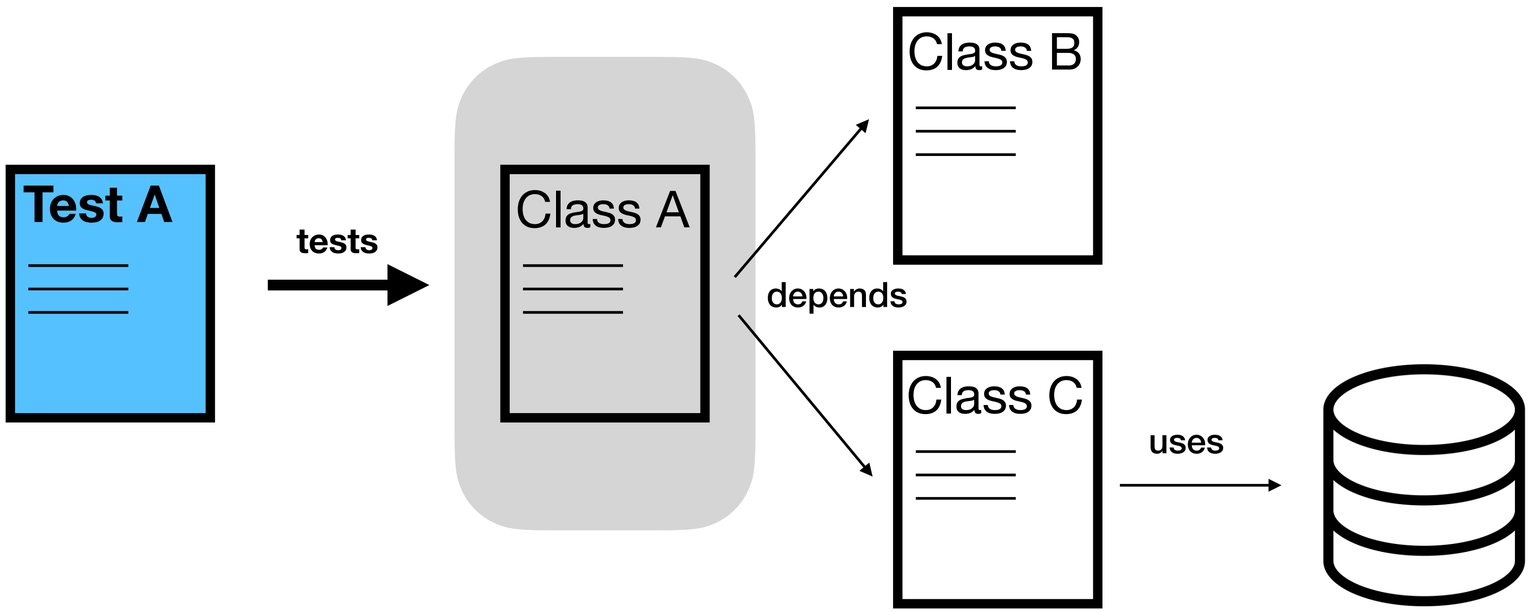
1. Less Debugging
2. Increased Confidence in Changes
3. Improved Documentation
4. Thoughtful Design
5. Fast, High Quality Software Releases

Test Scope

Unit Testing

A **unit** is an individual component of a system, such as a class or an individual method.

Testing units in isolation is called **unit testing**.



Unit Testing

A unit is an individual component of a system, such as a class or an individual method.

Testing units in isolation is called **unit testing**.

- **Fast**
- **Easy to control**
- **Easy to write**

- **Lack reality**
- **Cannot catch all bugs**
  (e.g. interactions with other components or services)

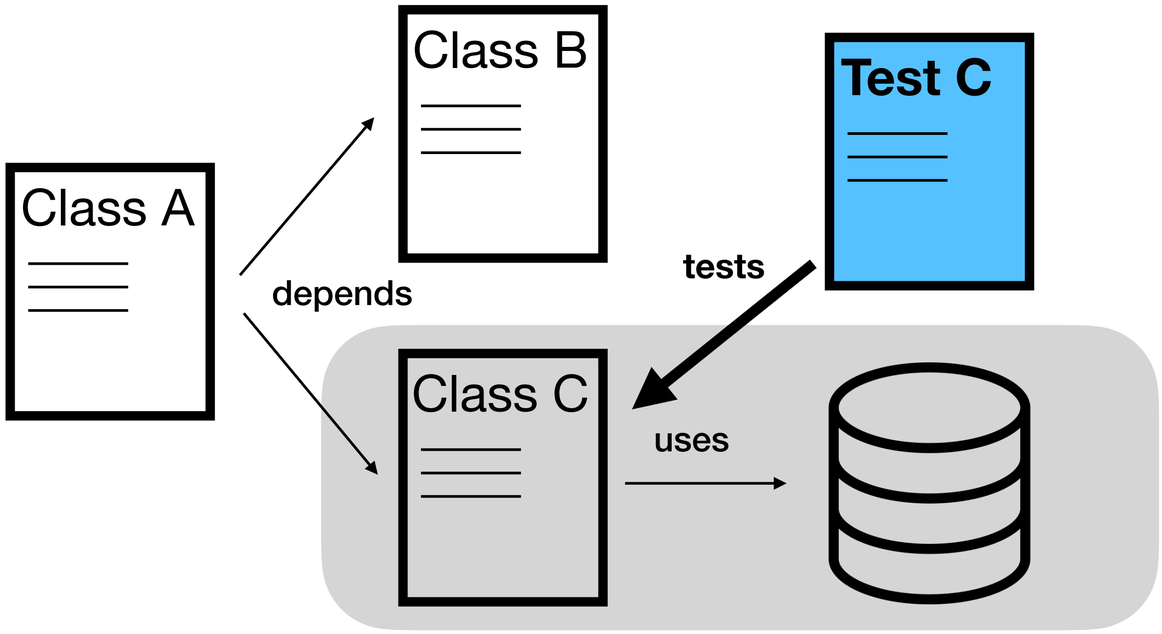
Unit tests are a very useful type of test but are often insufficient on their own.

Integration Tests

Testing in isolation is not enough. Sometimes code goes “beyond” the system’s borders and uses other (often external) components – for example, a database.

Integration tests test the integration between our code and that of external parties.

Example: Testing methods that access a database via SQL queries. Do our methods obtain the right data from the database?



Integration Tests

Testing in isolation is not enough. Sometimes code goes “beyond” the system’s borders and uses other (often external) components – for example, a database.

Integration tests test the integration between our code and that of external parties.

Example: Testing methods that access a database via SQL queries. Do our methods obtain the right data from the database?

- **Can capture integration bugs**
- **Less complex than writing system tests that goes through the entire system,** including components we do not care about

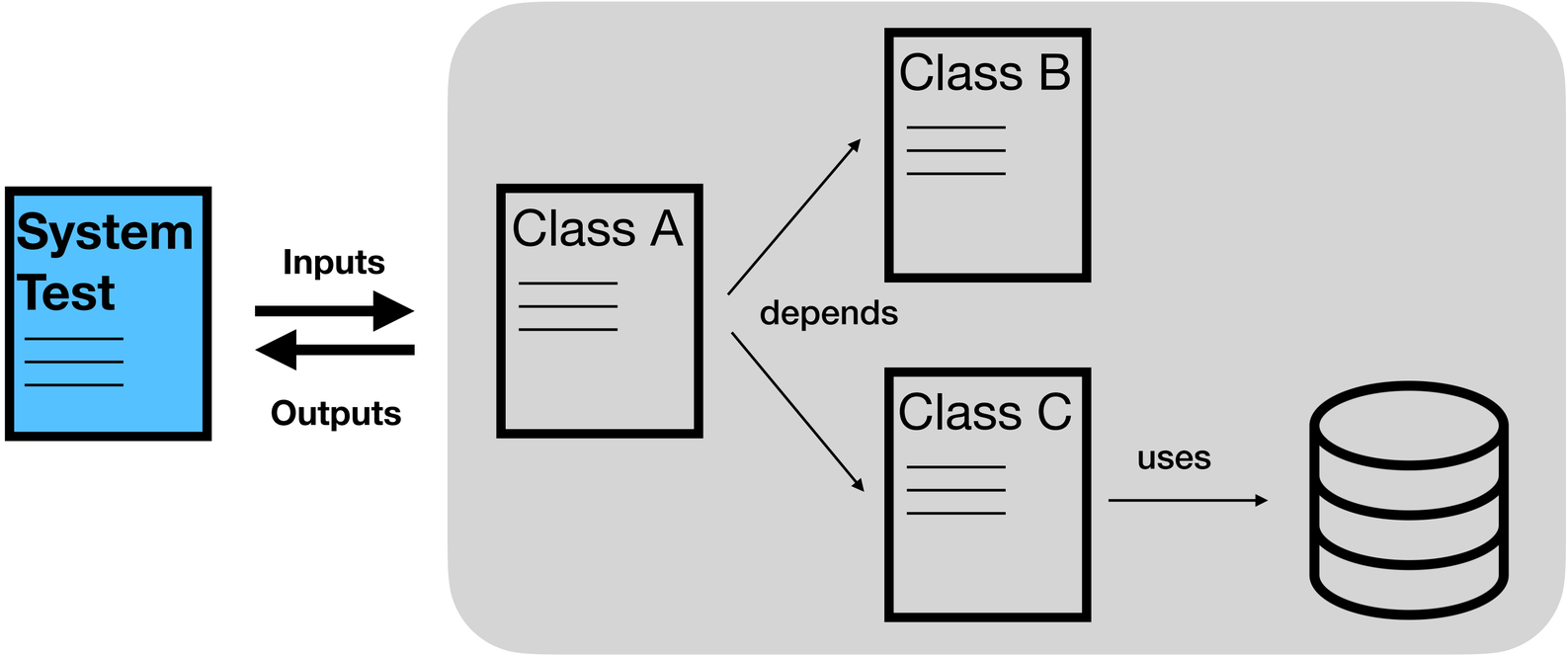
- **Hard to write**, for example:
  - Need to use an isolated DB instance
  - Put it into a state expected by the test
  - Reset the state afterwards

System Tests

To get a more realistic view of the software we should also perform **more realistic tests** with it – with all its database, front-end, and other components.

We do not care about how the system works from the inside.

We care that given certain inputs, certain outputs are provided by the system.



System Tests

To get a more realistic view of the software we should also perform more realistic tests with it – with all its database, front-end, and other components.

We do not care about how the system works from the inside.

We care that given certain inputs, certain outputs are provided by the system.

- **Realistic**
  (when the tests perform similarly to the end user, we can be more confident that the system will work correctly for all end users)

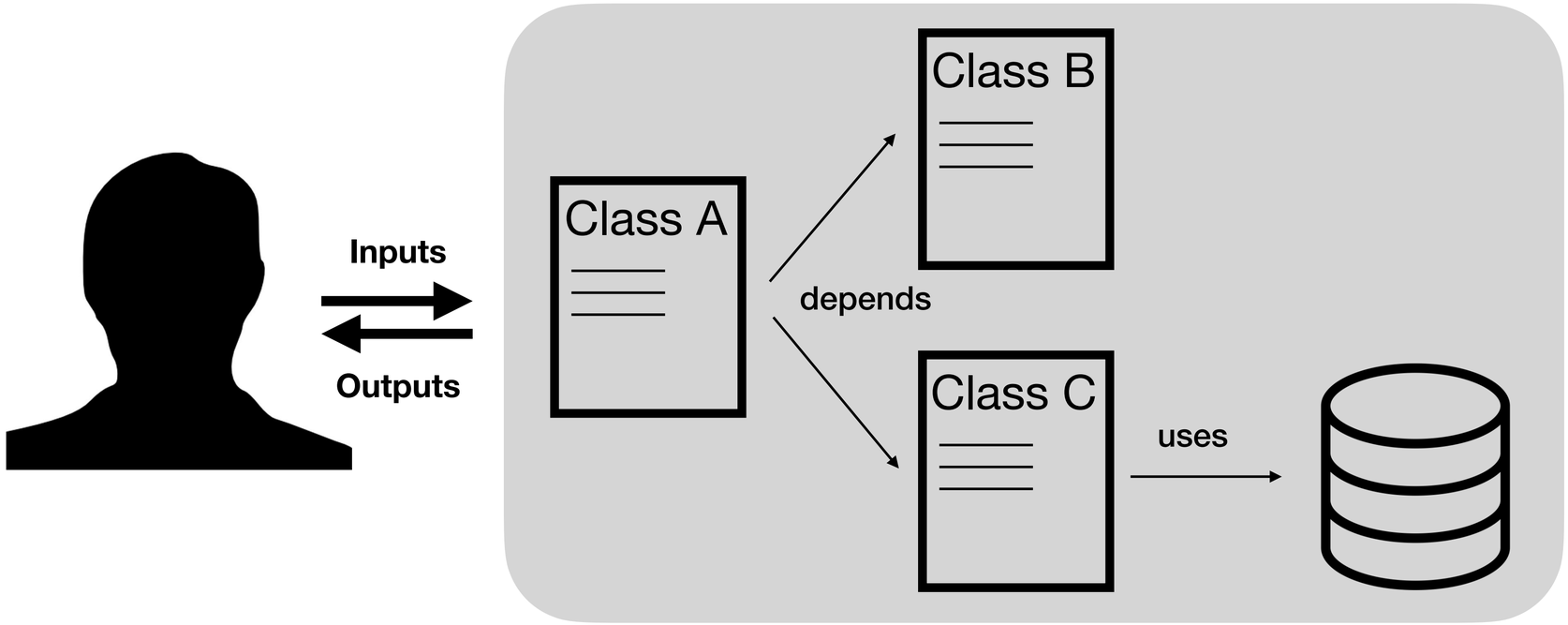
- **Slow!**
- **Hard to write**
  (lots of external services to account for)
- **(More) prone to Flakiness**

Manual Tests

Not everything can be tested easily in an automated fashion, particularly where there are qualitative judgements (e.g., the quality of a search engine's results).

Furthermore, we may need to **explore** real system behaviour to know what automated tests to write.

Manual tests are system tests performed manually by a human.



Manual Tests

Not everything can be tested easily in an automated fashion, particularly where there are qualitative judgements (e.g., the quality of a search engine's results).

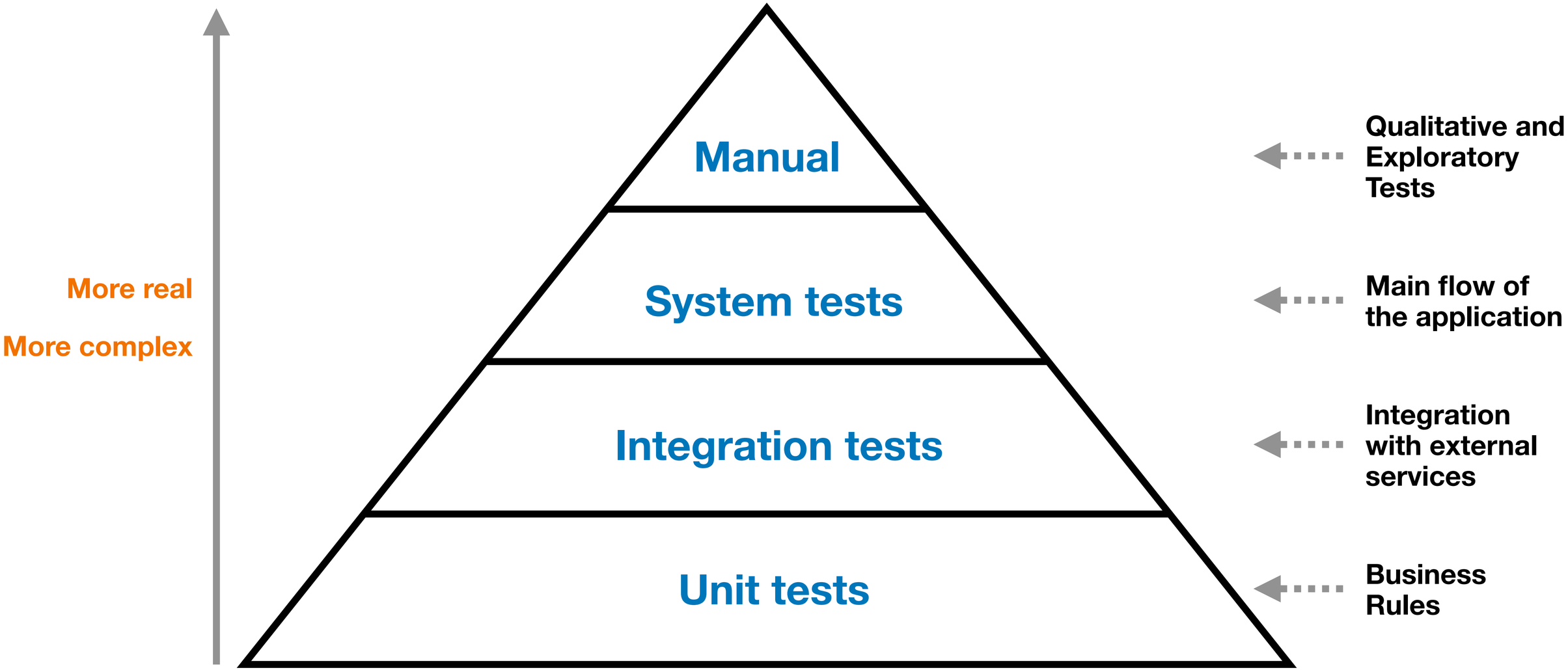
Furthermore, we may need to explore real system behaviour to know what automated tests to write.

Manual tests are system tests performed manually by a human.

- **Real**
  (The tester is acting as an end-user, actually using the system)

- **Time-consuming**
- **Difficult to reproduce**
- **Tedious**

The Test Triangle



What type of test am I?

🤔

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
YRJADQ

Enable answers by SMS

What type of test am I?

1. I test a web application. I load up the web page, automatically fill out forms, click on buttons, and check the resulting web page.

2. I check the result of a Java method given specific input values.

3. I am a series of values that a human uses to execute a program many times. I don't check the answers, I leave that to the human. I'm often written into tables or documents so that humans know how to reproduce me.

4. I am an automated test that interacts directly with code that uses a live RESTful API.